

# MDC-101 Flutter：Material Design 元件基本概念

來源：<https://codelabs.developers.google.com/codelabs/mdc-101-flutter?hl=zh-tw>

步驟數：6

為簡易 Flutter 應用程式建立登入頁面，瞭解使用 Material 元件的基本概念。

---

# 目錄

1. [簡介](#)
2. [設定 Flutter 開發環境](#)
3. [下載程式碼實驗室範例應用程式](#)
4. [新增 TextField 小工具](#)
5. [新增按鈕](#)
6. [恭喜！](#)

# 1. 簡介

## 什麼是 Material Design 和 Material Flutter 程式庫？

**Material Design** 是一套系統，可協助您打造大膽且美觀的數位產品。只要在一致的原則和元件下整合樣式、品牌、互動和動態效果，產品團隊就能發揮最大的設計潛力。

**Material Flutter 程式庫**包含實作 Material Design 元件設計的 Flutter 小工具 (簡稱 *MDC*)，可在應用程式和平台間提供一致的使用者體驗。隨著 Material Design 系統不斷演進，這些元件也會隨之更新，確保實作方式完全一致，並符合 Google 的前端開發標準。

在本程式碼研究室中，您將使用多個 Material Flutter 元件建構登入頁面。

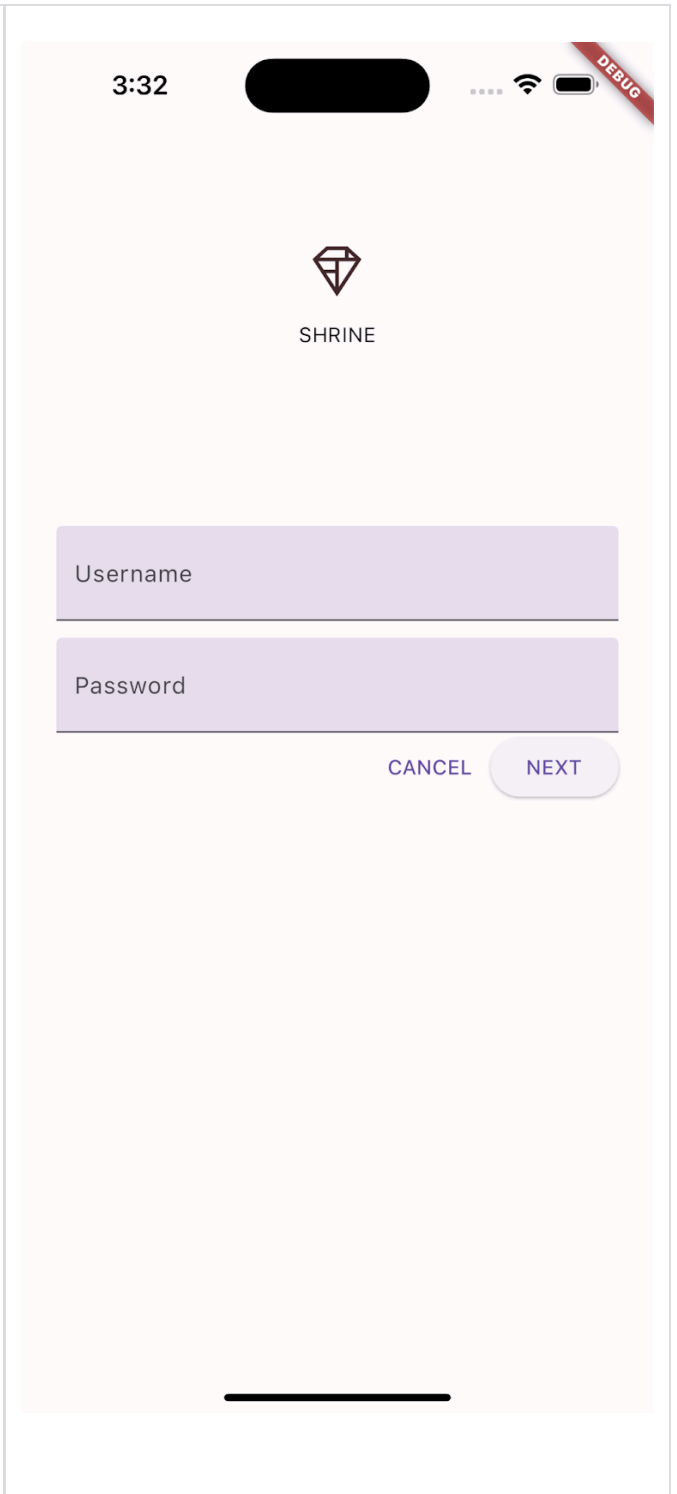
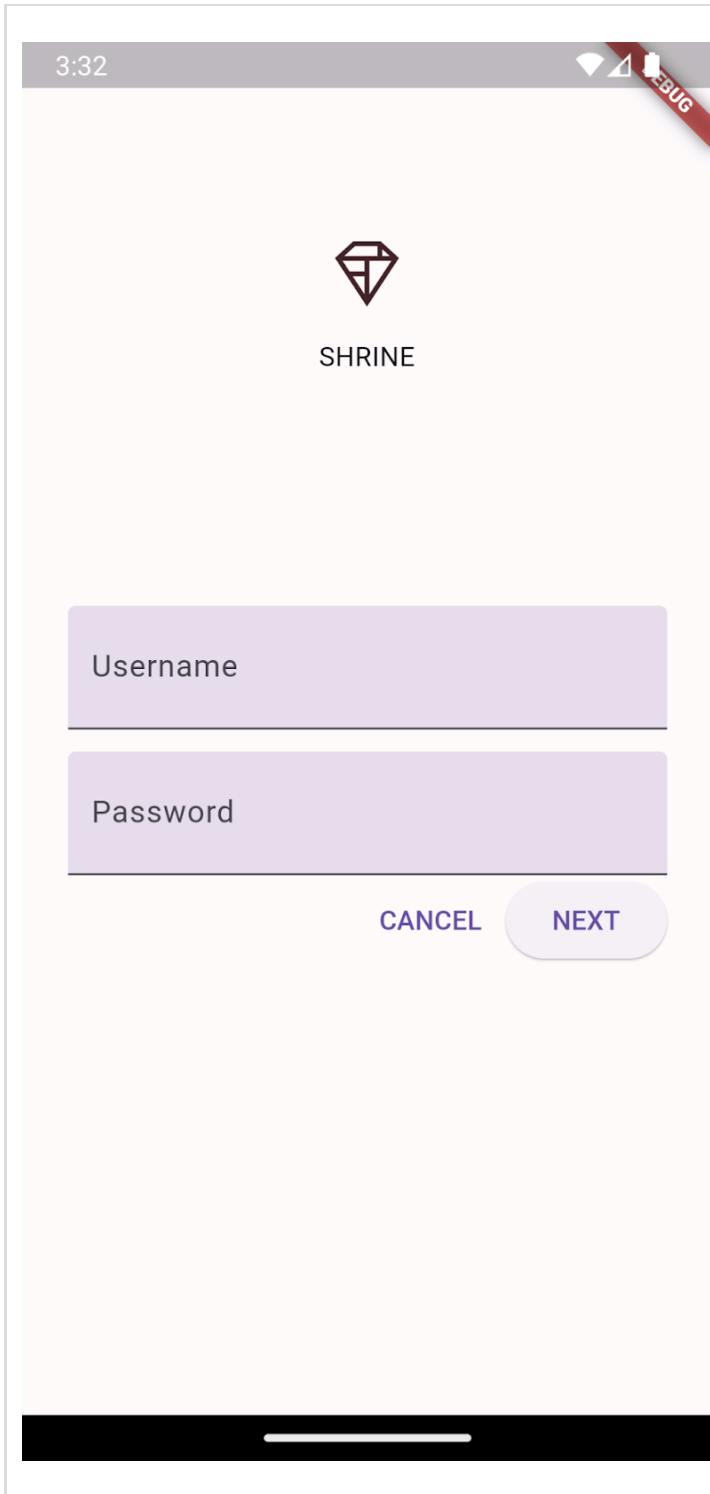
## 建構項目

本程式碼研究室是四個程式碼研究室的第一個，將引導您建構名為「Shrine」的應用程式，這是一個販售服飾和居家用品的電子商務應用程式。並說明如何使用 Material Flutter 自訂元件，反映任何品牌或風格。

在本程式碼研究室中，您將為 Shrine 建構登入頁面，其中包含：

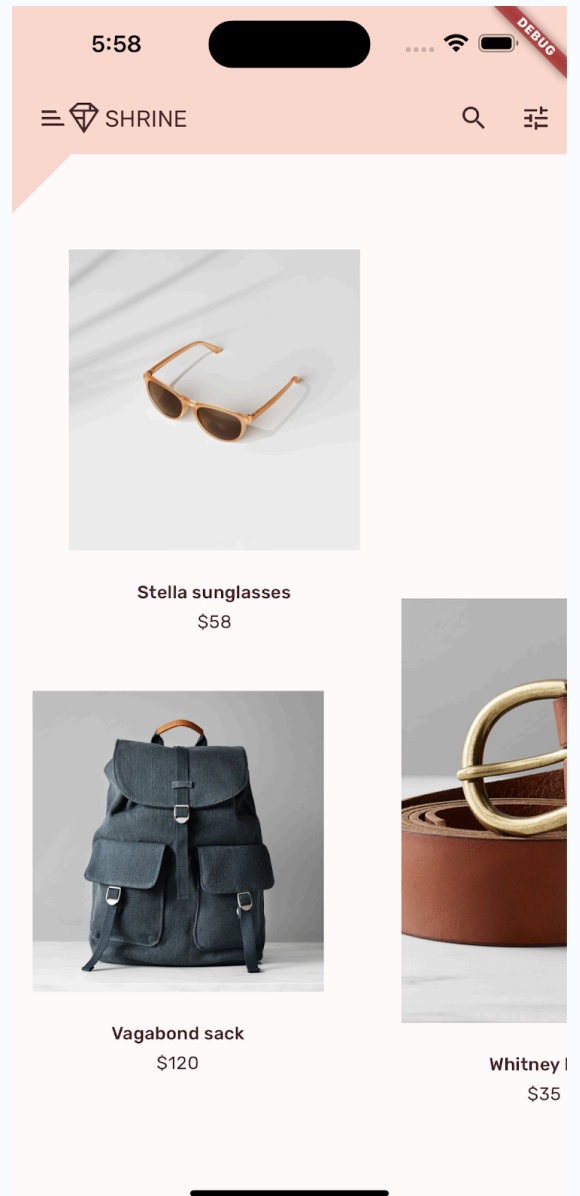
- Shrine 標誌的圖片
- 應用程式名稱 (Shrine)
- 兩個文字欄位，一個用於輸入使用者名稱，另一個用於輸入密碼
- 兩個按鈕

|         |     |
|---------|-----|
| Android | iOS |
|---------|-----|



本程式碼研究室是四個系列的第一個，將引導您為名為 Shrine 的產品建構應用程式。建議您依序完成所有程式碼研究室，逐步完成各項工作。相關程式碼研究室如下：

- [MDC-102 Flutter：Material Design 結構和版面配置](#)
- [MDC-103 Flutter：利用顏色、形狀、高度和類型建立 Material Design 主題](#)
- [MDC-104 Flutter：Material 進階元件](#)



## 本程式碼研究室中的 Material Flutter 元件和子系統

- 文字欄位
- 按鈕
- 墨水漣漪 (觸控事件的視覺回饋形式)

在本程式碼研究室中，您將使用 Material Flutter 提供的預設元件。您將在 [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題](#) 中，瞭解如何自訂這些屬性。

## 2. 設定 Flutter 開發環境

如要完成本實驗室，您需要兩項軟體：[Flutter SDK](#) 和[編輯器](#)。

您可以使用下列任一裝置執行程式碼研究室：

- 連線至電腦並設為開發人員模式的實體 [Android](#) 或 [iOS](#) 裝置。
  - [iOS 模擬器](#) (需要安裝 Xcode 工具)。
  - [Android Emulator](#) (需在 Android Studio 中設定)。
  - 瀏覽器 (偵錯時必須使用 Chrome)。
  - 以 [Windows](#)、[Linux](#) 或 [macOS](#) 電腦版應用程式的形式。您必須在要部署的平台上進行開發。因此，如要開發 Windows 桌面應用程式，您必須在 Windows 上開發，才能存取適當的建構鏈。如需作業系統專屬需求，請參閱 [docs.flutter.dev/desktop](https://docs.flutter.dev/desktop)。
-

步驟 3 · 約 0 分鐘

## 3. 下載程式碼實驗室範例應用程式

範例專案位於 `material-components-flutter-codelabs-101-starter/mdc_100_series` 目錄中。

### ...或從 GitHub 複製

如要從 GitHub 複製本程式碼研究室，請執行下列指令：

```
git clone https://github.com/material-components/material-components-flutter-codelabs.git
cd material-components-flutter-codelabs/mdc_100_series
git checkout 101-starter
```

如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

## 正確的分支版本

MDC-101 到 104 程式碼研究室會循序漸進，因此，當您完成 101 的程式碼後，該程式碼就會成為 102 的範例程式碼！程式碼會分散在不同分支中，您可以使用下列指令列出所有分支：

```
git branch --list
```

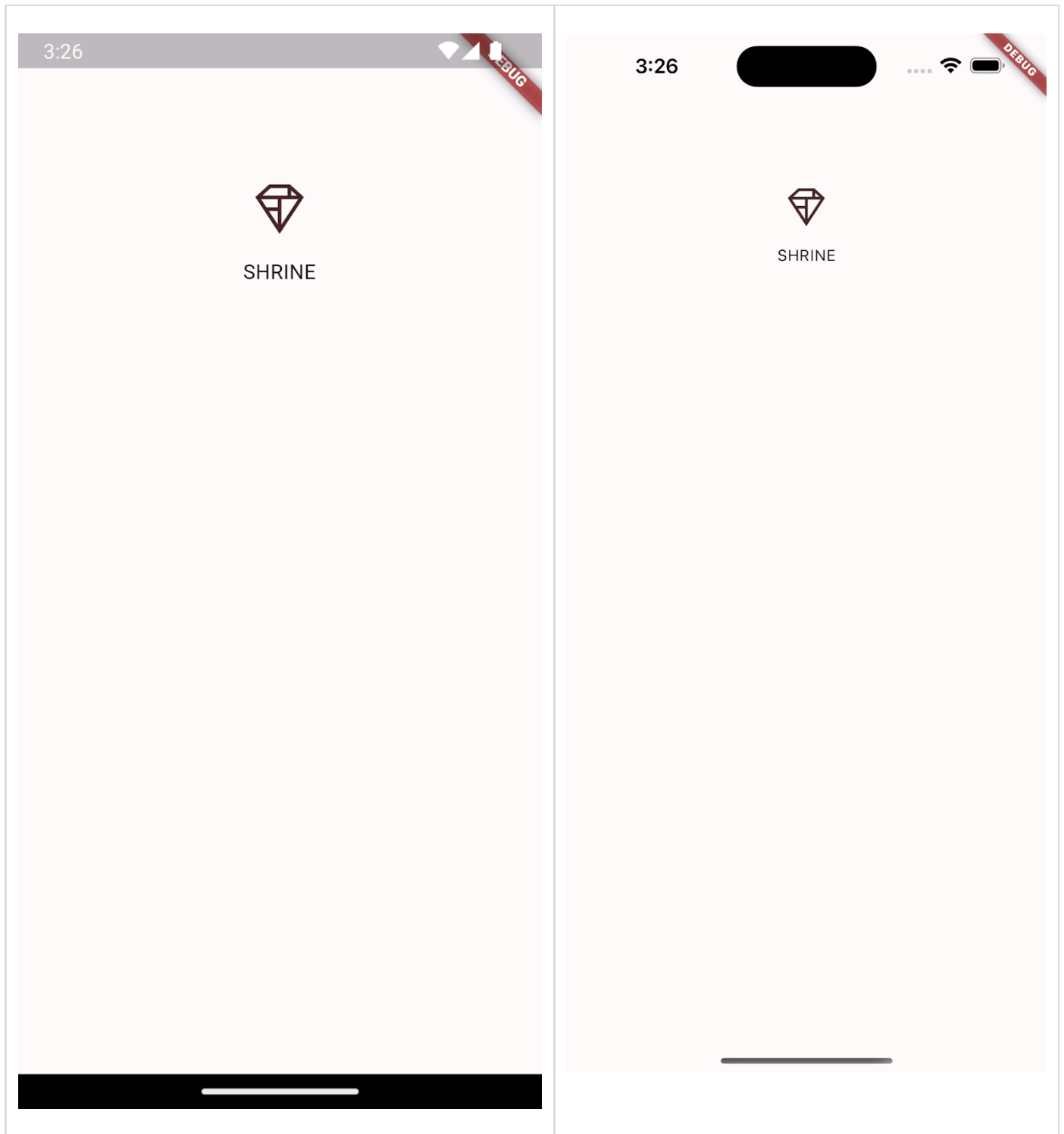
如要查看完成的程式碼，請切換至 `102-starter_and_101-complete` 分支。

## 開啟專案並執行應用程式

1. 在您選擇的編輯器中開啟專案。
2. 按照所選編輯器的「[開始使用：試用](#)」一節中的「執行應用程式」操作說明操作。

太棒了，裝置上應正在執行 Shrine 登入頁面的範例程式碼。您應該會看到 Shrine 標誌，以及標誌下方的「Shrine」名稱。

|         |     |
|---------|-----|
| Android | iOS |
|---------|-----|



我們來看看程式碼。

程式碼中的 TODO 包含數字 (101、102 等)，這些數字是指您要實作變更的程式碼研究室編號。

## 小工具 `login.dart`

開啟 `login.dart`。其中應包含下列項目：



```

import 'package:flutter/material.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({Key? key}) : super(key: key);

  @override
  _LoginPageState createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  // TODO: Add text editing controllers (101)
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.symmetric(horizontal: 24.0),
          children: <Widget>[
            const SizedBox(height: 80.0),
            Column(
              children: <Widget>[
                Image.asset('assets/diamond.png'),
                const SizedBox(height: 16.0),
                const Text('SHRINE'),
              ],
            ),
            const SizedBox(height: 120.0),
            // TODO: Remove filled: true values (103)
            // TODO: Add TextField widgets (101)
            // TODO: Add button bar (101)
          ],
        ),
      ),
    );
  }
}

```

請注意，其中包含 `import` 陳述式和兩個新類別：

- `import` 陳述式可讓您在這個檔案中使用 `Material` 程式庫。
- `LoginPage` 類別代表模擬器中顯示的整個頁面。
- `_LoginPageState` 類別的 `build()` 函式會控管 UI 中所有小工具的建立方式。

如要進一步瞭解 Flutter UI 的基本概念和小工具，可以參閱「[Flutter 小工具架構導覽](#)」或完成「[編寫您的第一個 Flutter 應用程式 - 第 1 部分](#)」程式碼研究室。

## 4. 新增 TextField 小工具

首先，我們會在登入頁面中加入兩個文字欄位，供使用者輸入使用者名稱和密碼。我們會使用 TextField 小工具，顯示浮動標籤並啟用觸控漣漪。

我們大幅升級了 Material 的文字欄位，以提升品牌彈性，並根據廣泛的使用者體驗 (UX) 研究進行改良，提供更優質的可用性和視覺設計。

如要進一步瞭解新的文字欄位，請參閱 [Material 指南文章](#)。

這個頁面主要採用 **ListView** 結構，可將子項排列在可捲動的資料欄中。讓我們在下方放置文字欄位。

### 新增 TextField 小工具

在 `const SizedBox(height: 120.0)` 後方新增兩個文字欄位和一個間隔。

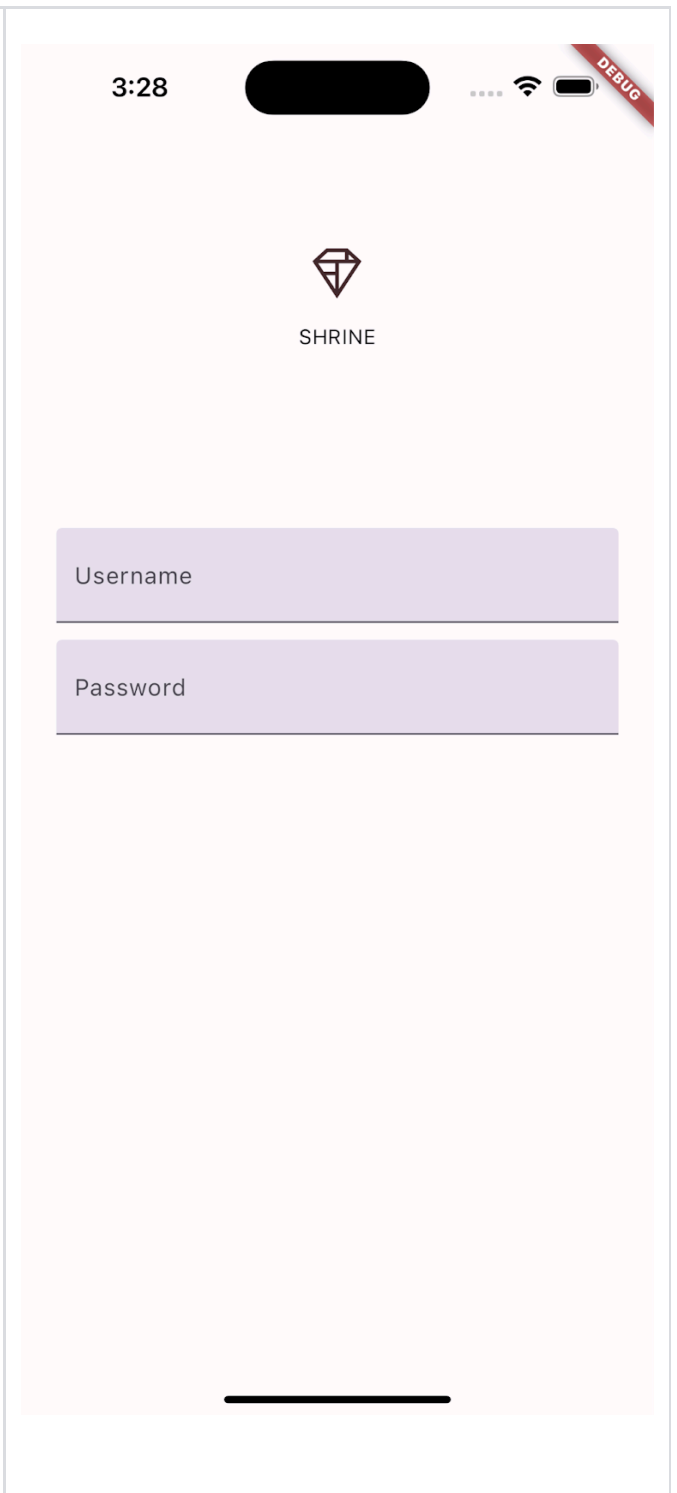
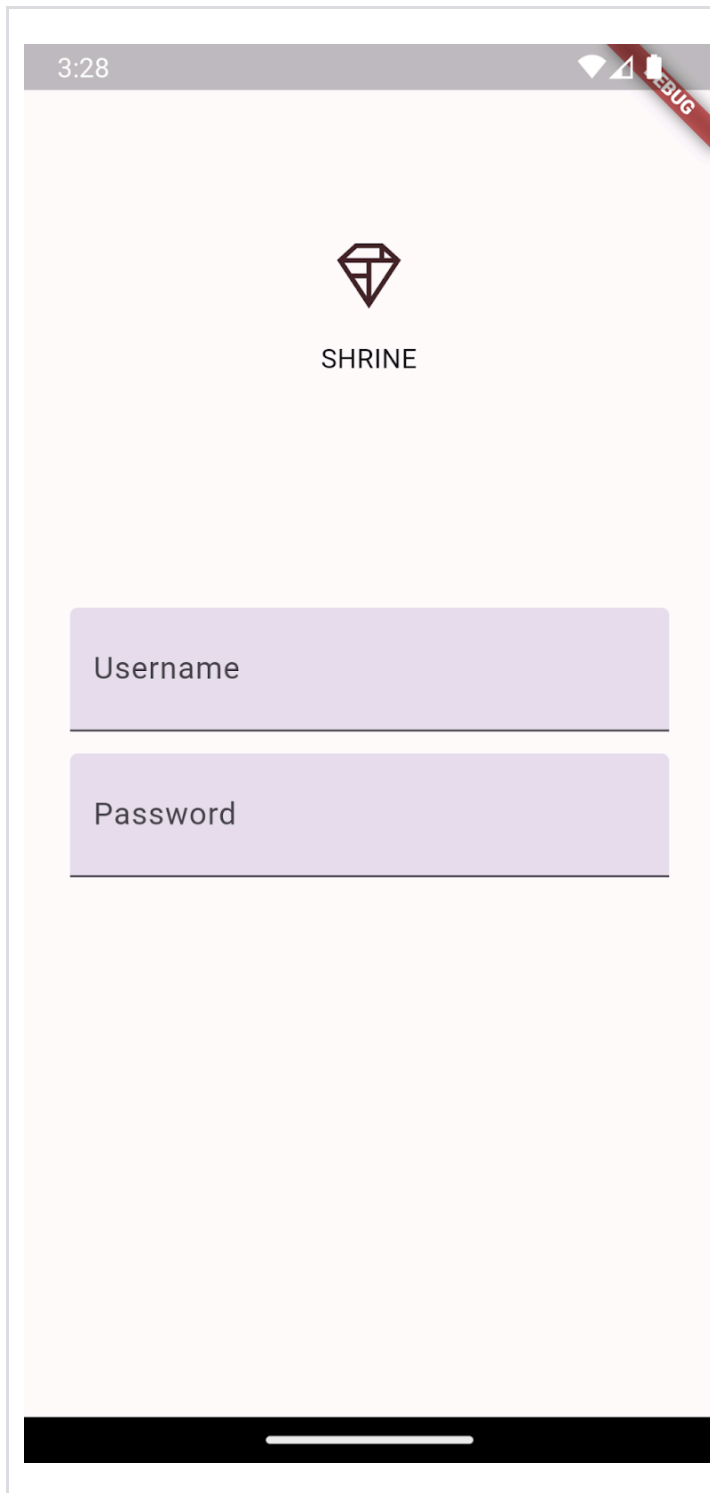
```
// TODO: Add TextField widgets (101)
// [Name]
TextField(
  decoration: const InputDecoration(
    filled: true,
    labelText: 'Username',
  ),
),
// spacer
const SizedBox(height: 120.0),
// [Password]
TextField(
  decoration: const InputDecoration(
    filled: true,
    labelText: 'Password',
  ),
  obscureText: true,
),
```

每個文字欄位都有 `decoration:` 欄位，可接受 **InputDecoration** 小工具。`filled:` 欄位表示文字欄位的背景已稍微填滿，方便使用者辨識文字欄位的輕觸或觸控目標區域。第二個文字欄位的 `obscureText: true` 值會自動將使用者輸入的內容替換為圓點，適合用於密碼。

儲存專案 (使用鍵盤快速鍵：command + s)，執行「熱重載」。

現在應該會看到一個頁面，其中有兩個文字欄位，分別是使用者名稱和密碼！請參閱浮動標籤動畫：

|         |     |
|---------|-----|
| Android | iOS |
|---------|-----|



TextField 的功能包括：

- 您可以輕鬆變更 TextField 小工具的外觀。請為裝飾欄位指定 InputDecoration 值。
- 文字欄位預設會顯示觸控回饋 (稱為漣漪或「墨水」)。
- FormField 是類似的小工具，具有在表單中嵌入欄位的特殊功能。
- [TextField 類別說明文件](#)

## 5. 新增按鈕

接著，我們會在登入頁面中新增兩個按鈕：「取消」和「下一步」。我們會使用兩種按鈕小工具：**TextButton** 和 **ElevatedButton**。

### 選擇文字按鈕或凸顯按鈕

為什麼不直接顯示兩個凸顯按鈕？每個按鈕類型都代表不同重要程度的動作。

版面配置應包含單一醒目按鈕。這樣一來，其他按鈕的重要性就會降低。這個醒目的按鈕代表我們最希望使用者採取的動作，以便在應用程式中繼續操作。

我們最不希望他們採取取消登入的動作。凸顯按鈕會以凸起的外觀吸引目光，因此應做為較重要的動作。相較之下，左側的純文字按鈕就顯得不那麼強調。

如要進一步瞭解按鈕階層，請參閱 Material 指南的「[按鈕](#)」一文。

## 新增 OverflowBar

在文字欄位後方，將 `OverflowBar` 新增至 `ListView` 的子項：

```
// TODO: Add button bar (101)
OverflowBar(
  alignment: MainAxisAlignment.end,
  // TODO: Add a beveled rectangular border to CANCEL (103)
  children: <Widget>[
    // TODO: Add buttons (101)
  ],
),
```

`OverflowBar` 會將子項排列成一行。

## 新增按鈕

接著在 `OverflowBar` 的 `children` 清單中新增兩個按鈕：

```
// TODO: Add buttons (101)
TextButton(
  child: const Text('CANCEL'),
  onPressed: () {
    // TODO: Clear the text fields (101)
  },
),
// TODO: Add an elevation to NEXT (103)
// TODO: Add a beveled rectangular border to NEXT (103)
ElevatedButton(
  child: const Text('NEXT'),
  onPressed: () {
    // TODO: Show the next page (101)
  },
),
```

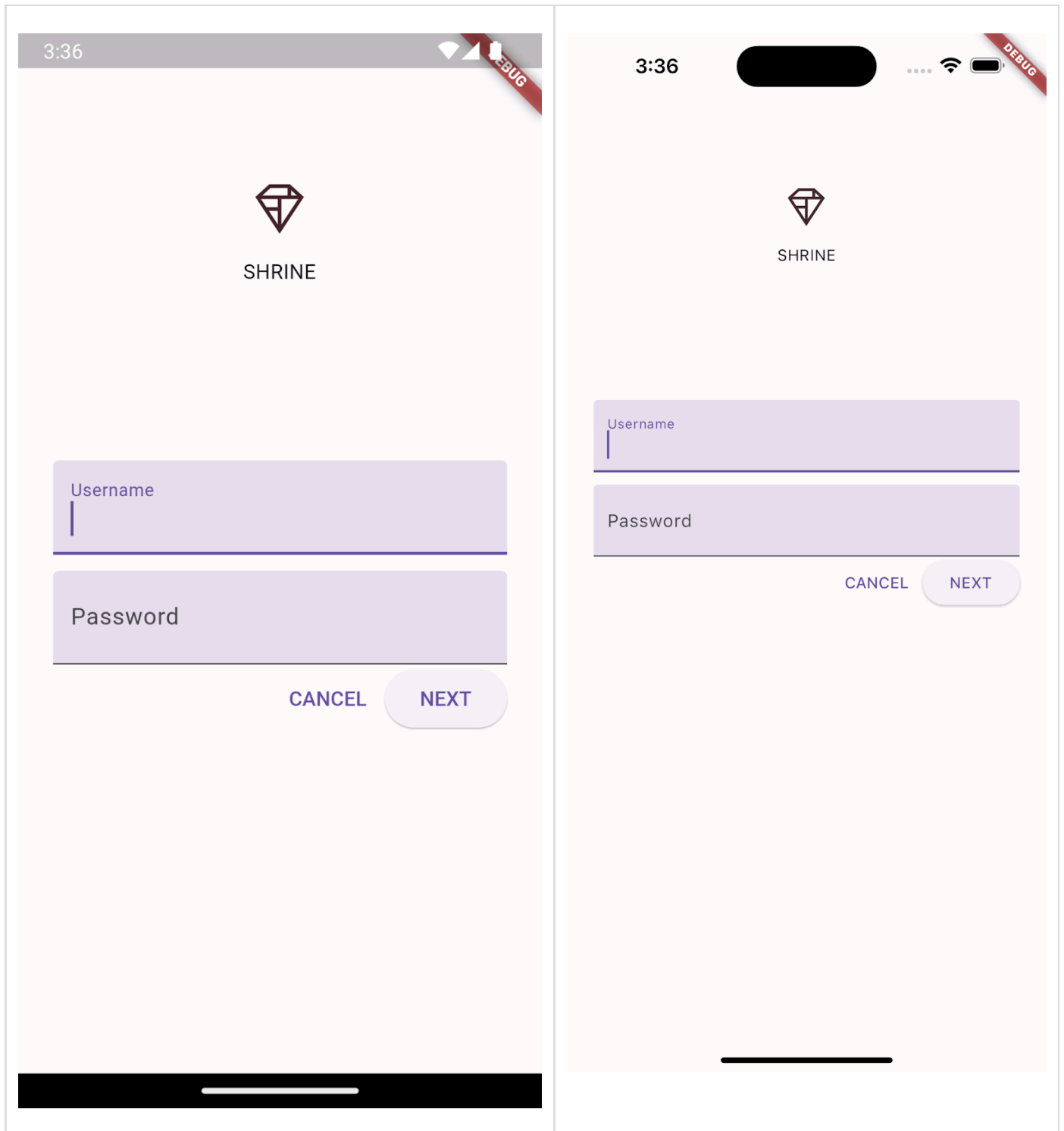
## 為什麼 onPressed: 欄位有空白區塊？

如果我們傳遞空值，按鈕就會停用。這樣一來，我們就無法取得觸控回饋，也無法瞭解啟用行為。使用空白區塊可避免遭到停用。

如要進一步瞭解按鈕及其互動方式，請參閱「[在應用程式中加入互動功能](#)」。

儲存專案。最後一個文字欄位下方會顯示兩個按鈕：

|         |     |
|---------|-----|
| Android | iOS |
|---------|-----|



`OverflowBar` 會為您處理版面配置工作。按鈕會水平放置，並排顯示。

輕觸按鈕會啟動墨水漣漪動畫，但不會觸發其他動作。我們在匿名 `onPressed:` 函式中新增功能，讓取消按鈕清除文字欄位，下一個按鈕則會關閉畫面：

## 新增 `TextEditingController`

為了能夠清除文字欄位的值，我們會新增 `TextEditingController` 來控制文字。

在 `_LoginPageState` 類別的宣告正下方，將控制器新增為 `final` 變數。

```
// TODO: Add text editing controllers (101)
final _usernameController = TextEditingController();
final _passwordController = TextEditingController();
```

在第一個文字欄位的 `controller:` 欄位中，設定 `_usernameController`：

```
// TODO: Add TextField widgets (101)
// [Name]
TextField(
  controller: _usernameController,
```

在第二個文字欄位的 `controller:` 欄位中，現在請設定 `_passwordController`：

```
// TODO: Add TextField widgets (101)
// [Password]
TextField(
  controller: _passwordController,
```

## 編輯 onPressed

在 `TextButton` 的 `onPressed:` 函式中新增指令，清除每個控制器：

```
// TODO: Clear the text fields (101)
_usernameController.clear();
_passwordController.clear();
```

儲存專案。現在在文字欄位中輸入內容後，按下「取消」會再次清除每個欄位。

這個登入表單狀態良好！現在要讓使用者進入 Shrine 應用程式的其餘部分。

## 流行

如要關閉這個檢視畫面，我們需要從導覽堆疊中彈出 (或移除) 這個頁面 (Flutter 稱為路徑)。

**Navigator** 會維護一個路由堆疊，就像 iOS 上的 `UINavigationController` 一樣。*推送*路由會將其置於堆疊頂端。*彈出*堆疊會移除最近新增的路徑。在應用程式的 `app.dart` 中，呼叫 `initialRoute: '/login'`，會將登入畫面新增至 `Navigator`。

如要瞭解路線和導航功能，請參閱「[導航基本概念](#)」。

在 `ElevatedButton` 的 `onPressed:` 函式中，從 `Navigator` 彈出最近的路線：

```
// TODO: Show the next page (101)
Navigator.pop(context);
```

最後，開啟 `home.dart`，並在 `Scaffold` 中將 `resizeToAvoidBottomInset` 設為 `false`：

```
return Scaffold(  
  // TODO: Add app bar (102)  
  // TODO: Add a grid view (102)  
  body: Center(  
    child: Text('You did it!'),  
  ),  
  // TODO: Set resizeModeAvoidBottomInset (101)  
  resizeModeAvoidBottomInset: false,  
);
```

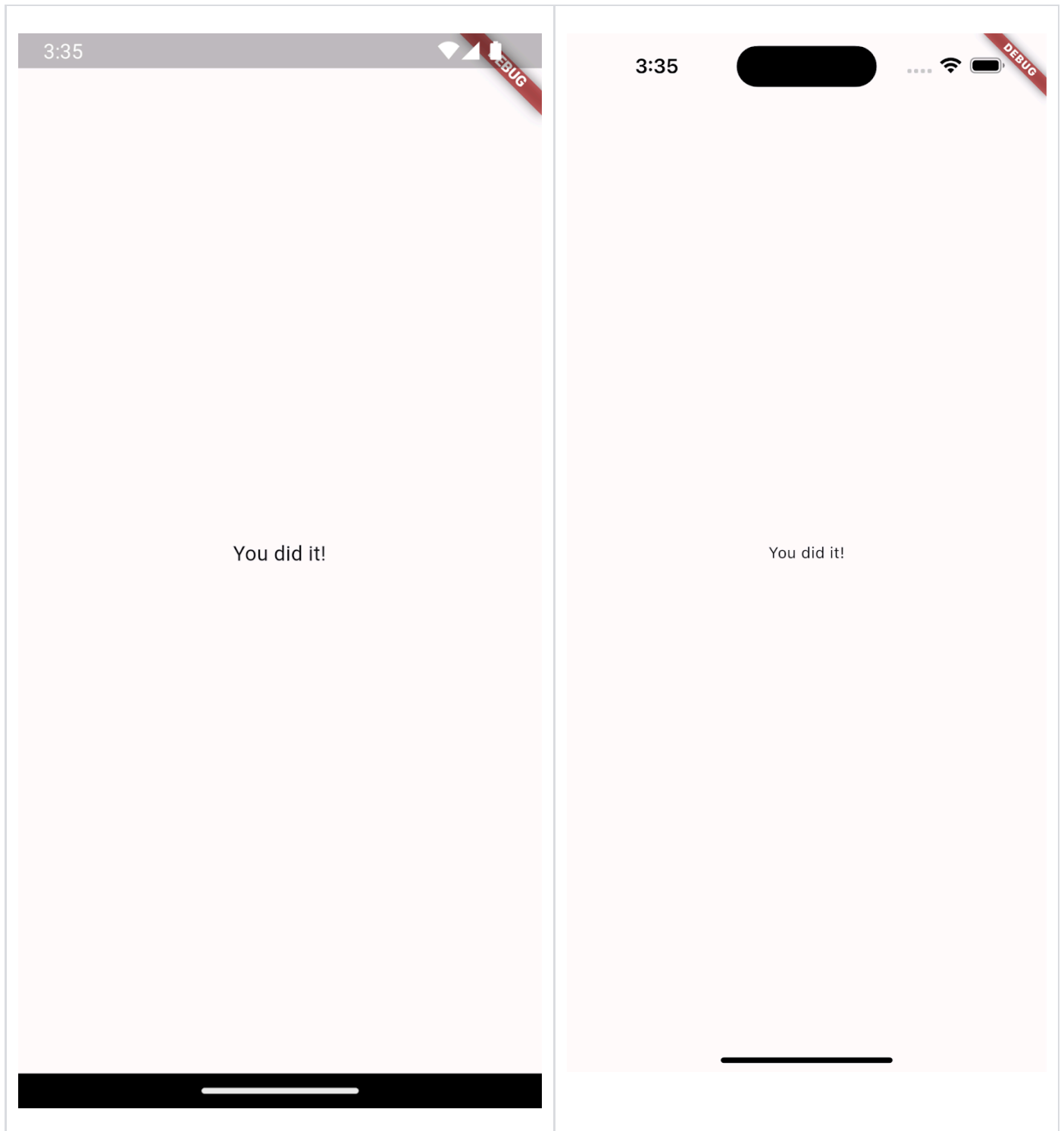
這麼做可確保鍵盤不會改變首頁或小工具的大小。

大功告成！儲存專案。然後按一下「下一步」。

你們成功了！

|         |     |
|---------|-----|
| Android | iOS |
|---------|-----|





這個畫面是下一個程式碼研究室的起點，您將在 MDC-102 中處理這個畫面。

關於按鈕：

- 除了 `TextButton` 和 `ElevatedButton`，還有 `OutlinedButton`、`FloatingActionButton` `IconButton` 等等。

## 6. 恭喜！

我們新增了文字欄位和按鈕，幾乎不必考慮版面配置程式碼。Flutter 適用的 Material 元件提供許多樣式，幾乎不費吹灰之力就能放置在畫面上。

完成的 MDC-101 應用程式位於 `102-starter_and_101-complete` 分支中。

您可以根據該分支版本中的應用程式測試應用程式版本。

## 後續步驟

文字欄位和按鈕是 Material 系統的兩個核心元件，但還有更多元件！您也可以在 [Material Components 小工具目錄](#) 中探索其餘部分。

或者，您也可以前往 [MDC-102：Material Design 結構和版面配置](#)，瞭解 MDC-102 中涵蓋的 Flutter 元件。

---